DATE:

DATA ANALYSIS

FOR BEGINNERS

Data Analysis is the process of collecting, cleaning, organizing, and studying data to discover useful information and make better decisions.

It helps us understand what happened, why it happened, and what may happen next.

Why do we use Data Analysis?

@Mastercode.sagar

- Find patterns and trends
- Solve problems using data
- Make informed decisions
- Present insights clearly

Simple Example :

A shop studies monthly sales data to find its best-selling product.

Basic Data Analysis Flow :

Collect → Clean → Analyze → Visualize → Decide

By: Master Code Sagar

DATE :

Types of Data

Data can be classified into different types based on its nature, structure and the kind of information it represents.

1. Structured Data :

- Organized in a fixed format
- Usually stored in tables (rows and columns)
- Easily analyzed using tools like Excel, SQL, Python
- Examples: Student records, sales data, employee details

ID	Name	Age	City
1	Rahul	21	Delhi
2	Priya	20	Mumbai
3	Aman	22	Jaipur

← This is structured data (tabular form)

2. Unstructured Data :

- Does not have a fixed format
- More difficult to analyze
- Examples: Text files, images, videos, social media posts, emails, PDFs etc.


Text file


Image


Video


Email

3. Semi-structured Data :

- Does not follow a strict table format but has some structure or tags.
- Examples: JSON, XML, HTML data, log files.

```
{
  "name" : "Sagar",
  "age" : 19,
  "city" : "Delhi"
}
```

→ This is semi-structured data (JSON format)

@ Mastercode sagar

Data Analysis Process

Data Analysis follows a step-by-step process to turn raw data into useful information. Each step helps us understand the data better and make informed decisions.

1. Define the Problem
   - Understand the goal and what question we want to answer.
   - Example : Find out why sales are decreasing.
2. Collect Data
   - Gather data from reliable sources.
   - Sources : Excel files, databases, websites, APIs, surveys, etc.
3. Clean the Data
   - Remove errors, missing values and duplicate data.
   - Make the data consistent and properly formatted.
4. Explore the Data
   - Understand the structure of the data.
   - Find patterns, trends and relationships.
   - Use summary statistics and visualizations.
5. Analyze the Data
   - Apply different analysis techniques.
   - Use statistics, Python, Excel or SQL to find useful insights.
6. Visualize the Results
   - Create charts and graphs to represent the findings.
   - Helps in easy understanding and communication.
7. Draw Conclusions
   - Interpret the results and answer the original question.
   - Suggest possible actions based on the insights.

Problem Define → Collect Data → Clean Data → Explore Data → Analyze Data → Visualize Results → Conclusion Draw

DATE :

Tools Used in Data Analysis

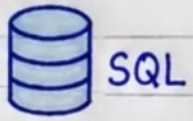
There are many tools available for data analysis. Each tool has its own purpose and is used at different stages of the analysis process.

1. Microsoft Excel



- Most popular tool for beginners.
- Useful for data cleaning, analysis and visualization.
- Provides formulas, pivot tables and charts.
- Suitable for small to medium sized datasets.

2. SQL (Structured Query Language)



- Used to work with relational databases.
- Helps in storing, querying and managing large amounts of data.
- Commonly used in companies for data analysis.
- Example: MySQL, PostgreSQL, SQL Server.

3. Python



- Powerful programming language for data analysis.
- Libraries like Pandas, NumPy, Matplotlib, Seaborn make analysis easy.
- Useful for data cleaning, analysis, visualization and machine learning.
- Suitable for small and large datasets.

4. R



- Specially designed for statistical analysis.
- Provides many packages for data visualization.
- Widely used in research and academia.
- Good for advanced statistical analysis.

5. Power BI



- Business intelligence tool by Microsoft.
- Used for interactive dashboards and reports.
- Helps in visualizing data in a simple and effective way.
- Commonly used in companies for reporting and decision making.

6. Google Sheets



- Web-based tool similar to Excel.
- Useful for collaboration and basic data analysis.
- Easy to share and access from anywhere.

DATE:

Introduction to Microsoft Excel

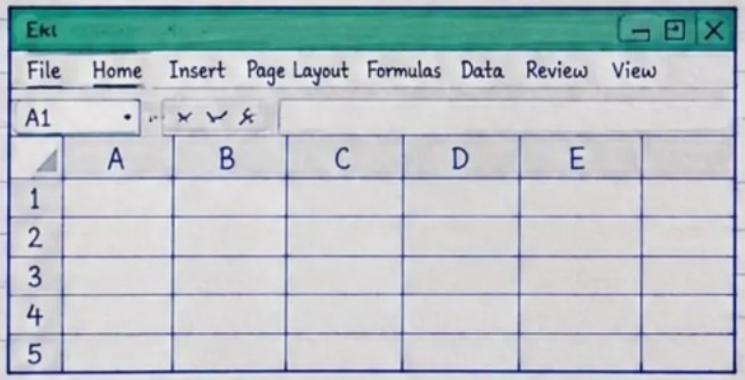
FOR DATA ANALYSIS

Microsoft Excel is a spreadsheet program developed by Microsoft. It is widely used for storing, organizing, analyzing and visualizing data.

1. What is a Spreadsheet ?

- A spreadsheet is a grid of rows and columns used to store data.
- Each cell can contain text, numbers or formulas.
- It helps in performing calculations and analysis easily.
- Files in Excel are called Workbooks (extension .xlsx).

2. Excel Interface (Main Parts)



- Title Bar - shows file name
- Ribbon - contains tools and features
- Formula Bar - to enter or edit data
- Cells - where data is stored
- Rows - numbered (1, 2, 3, ...)
- Columns - lettered (A, B, C, ...)
- Sheet - each workbook can have multiple sheets.

3. Basic Operations

- Enter data : Click on a cell and type the value.
- Edit data : Double click on the cell.
- Copy-Paste : Ctrl + C (copy), Ctrl + V (paste).
- Undo : Ctrl + Z
- Save file : Ctrl + S

4. Commonly Used Formulas

- SUM() - Adds numbers =SUM(A1:A5)
- AVERAGE() - Finds average =AVERAGE(A1:A5)
- COUNT() - Counts number of cells =COUNT(A1:A5)
- MAX() - Finds the largest value =MAX(A1:A5)
- MIN() - Finds the smallest value =MIN(A1:A5)

DATE:

Excel for Data Analysis

Excel provides many powerful features that help us to analyze, summarize and visualize data easily.

1. Working with Data

- Enter data in rows and columns.
- Keep the first row as a header.
- Use proper column names.
- Avoid blank rows and unnecesary spaces.
- Convert data range into a Table for better analysis.

	A	B	C	D
1	Name	Age	City	Sales
2	Rahul	21	Delhi	1200
3	Priya	20	Mumbai	1500
4	Aman	22	Jaipur	1100

2. Sorting and Filtering

- Sorting arranges data in ascending or descending order.
- Filtering helps to show only the data that meets a condition.
- Useful for analyzing large datasets.

A↓Z Sort	Filter	Sort A to Z Sort Z to A Filter by Value

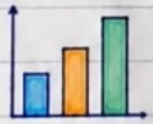
3. Conditional Formatting

- Highlights important data using colors and rules.
- Helps to quickly find patterns, trends or outliers.
- Examples : Highlight values greater than 1000, duplicate values, etc.

Product	Sales	Status
Mouse	800	Low
Keyboard	1500	High
Monitor	2200	High
USB	600	Low

4. Charts for Data Visualization

- Charts help to represent data in a visual form.
- Common charts : Column chart, Bar chart, Line chart, Pie chart, etc.
- Useful for comparing data and finding trends.


Column Chart


Bar Chart


Line Chart


Pie Chart

5. Pivot Table (Powerful Feature)

- Used to quickly summarize large amounts of data.
- Helps to find totals, averages, counts, etc.
- Very useful for real-world data analysis.

City	Total Sales
Delhi	1200
Mumbai	1500
Jaipur	1100
Grand Total	3800

DATE:

SQL Queries with Examples

(FOR DATA ANALYSIS)

Let's see some common SQL queries with examples to understand how SQL is used in real data analysis.

1. SELECT - Retrieve data from a table

- Used to fetch data from a table.

Query :

```
SELECT * FROM Student;
```

Output :

ID	Name	Age	City
1	Rahul	21	Delhi
2	Priya	20	Mumbai
3	Aman	22	Jaipur

2. SELECT specific columns

- Used to fetch only required columns.

Query :

```
SELECT Name, Age FROM Student;
```

Output :

Name	Age
Rahul	21
Priya	20
Aman	22

3. WHERE - Filter data

- Used to fetch records that meet a condition.

Query :

```
SELECT * FROM Student
WHERE Age > 21;
```

Output :

ID	Name	Age	City
3	Aman	22	Jaipur
5	Sagar	23	Indore

4. ORDER BY - Sort data

- Used to sort records in ascending (ASC) or descending (DESC) order.

Query :

```
SELECT * FROM Student
ORDER BY Age ASC;
```

Output :

ID	Name	Age	City
2	Priya	20	Mumbai
1	Rahul	21	Delhi
3	Aman	22	Jaipur
5	Sagar	23	Indore

5. LIMIT - Retrieve limited number of records

- Used to get only a specific number of rows.

Query :

```
SELECT * FROM Student
LIMIT 3;
```

Output :

ID	Name	Age	City
1	Rahul	21	Delhi
2	Priya	20	Mumbai
3	Aman	22	Jaipur

6. COUNT , AVG , SUM , MAX , MIN

- Used to perform basic calculations.

Query examples :

```
SELECT COUNT(*) FROM Student;
SELECT AVG(Age) FROM Student;
SELECT MAX(Age) FROM Student;
SELECT MIN(Age) FROM Student;
```

Output :

Query	Result
COUNT(*)	5
AVG(Age)	21.6
MAX(Age)	23
MIN(Age)	20

DATE:

Data Cleaning in Python using Pandas (FOR DATA ANALYSIS)

Data cleaning is an important step in data analysis. It helps to remove errors, handle missing values and convert data into a proper format for analysis.

1. Checking for Missing Values

- We can use isnull() to check for missing values.
- It returns True for missing values and False for valid data.

```
import pandas as pd
df = pd.read_csv("student.csv")
print(df.isnull())
print(df.isnull().sum())
```

```
# check missing values
```

Output (df.isnull().sum()):

Name	Age	City
0	1	2

2. Handling Missing Values

- We can handle missing values by:
  - fillna() – fill with a specific value
  - dropna() – remove rows with missing values

```
df.fillna(0)                          # fill with 0
df["Age"].fillna(df["Age"].mean())    # fill with mean value
df.dropna()                           # remove rows with missing values
```

Example :

Name	Age	City
Rahul	21	Delhi
Priya	NaN	Mumbai
Aman	22	NaN

3. Removing Duplicate Rows

- Duplicates can affect analysis.
- We can remove them using drop_duplicates().

```
df.duplicated()          # check duplicate rows
df.drop_duplicates()     # remove duplicate rows
```

Before :

Name	Age	City
Rahul	21	Delhi
Rahul	21	Delhi
Priya	20	Mumbai

After drop_duplicates():

Name	Age	City
Rahul	21	Delhi
Priya	20	Mumbai

4. Changing Data Types

- Sometimes data is in wrong format.
- We can change data type using astype().

```
df["Age"] = df["Age"].astype(int)     # convert to integer
df["Name"] = df["Name"].astype(str)   # convert to string
print(df.dtypes)                      # check data types
```

5. Renaming Columns

- We can rename columns for better understanding.

```
df.rename(columns={'Name':'Student_Name',
                   'Age':'Student_Age'}, inplace=True)
print(df.columns)
```

Output :

Student_Name	Student_Age	City
Rahul	21	Delhi
Priya	20	Mumbai

6. Dropping Unnecessary Columns

- We can drop columns that are not needed.

```
df.drop(columns=["City"], inplace=True)
```

Output :

Name	Age
Rahul	21
Priya	20

DATE:

Exploratory Data Analysis (EDA)

(IN PYTHON)

Exploratory Data Analysis (EDA) is the process of analyzing and visualizing data to understand its main characteristics, find patterns, detect outliers and get useful insights before building a model.

1. Loading Required Libraries

- We will use Pandas for data handling.
- Matplotlib and Seaborn for data visualization.
- Let's load the libraries and dataset.

```
import pandas as pd                    # for data handling
import matplotlib.pyplot as plt        # for basic plots
import seaborn as sns                  # for better visualizations
df = pd.read_csv("student.csv")        # load dataset
```

Dataset Example :

Name	Age	Marks	City
Rahul	21	85	Delhi
Priya	20	78	Mumbai
Aman	22	92	Jaipur
Sagar	23	88	Indore

2. Understanding the Dataset

- We can use basic functions to get information about the data.

```
df.head()          # first 5 rows
df.info()          # summary of dataset
df.describe()      # statistical summary
df.shape           # number of rows and columns
df.columns         # column names
```

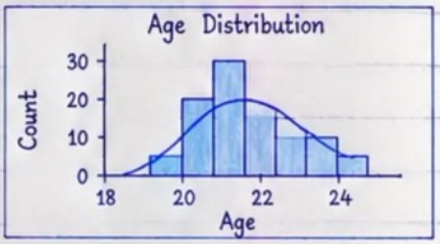
Example Output (df.describe()):

	Age	Marks
count	100	100
mean	21.5	78.4
std	1.3	10.2
min	19	50
25%	20	70
50%	21	80
75%	23	88
max	25	98

3. Univariate Analysis

- Used to analyze a single variable at a time.
- We can use histograms, bar plots etc.

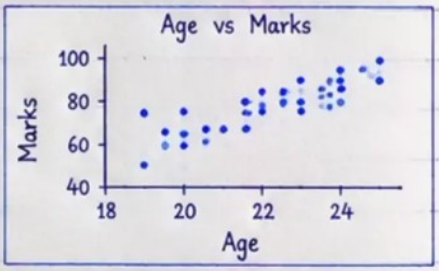
```
sns.histplot(df['Age'], kde=True)      # histogram for Age
plt.title('Age Distribution')          # show the plot
plt.show()
```



4. Bivariate Analysis

- Used to analyze the relationship between two variables.
- We can use scatter plot, line plot, bar plot etc.

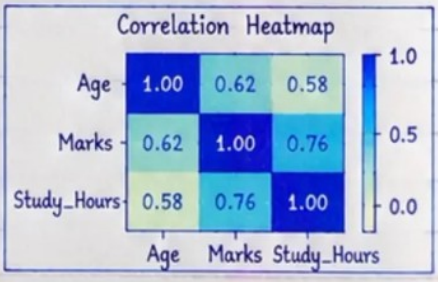
```
sns.scatterplot(x='Age', y='Marks', data=df)  # scatter plot
plt.title('Age vs Marks')                     # set title
plt.xlabel('Age')                             # x-axis label
plt.ylabel('Marks')                           # y-axis label
plt.show()
```



5. Multivariate Analysis

- Used to analyze more than two variables.
- We can use pairplot or heatmap.

```
sns.heatmap(df.corr(numeric_only=True),
    annot=True, cmap='YlGnBu')
plt.title("Correlation Heatmap")       # correlation heatmap
plt.show()                             # show the plot
```



6. Key Insights from EDA

- Helps to understand data distribution, trends and relationships.
- Detects missing values, outliers and important patterns.
- Useful for feature selection and model building.

DATE:

Data Visualization using Matplotlib (IN PYTHON)

Matplotlib is a popular Python library used for data visualization. It helps us to create different types of graphs and charts to understand data easily.

1. Installing and Importing Matplotlib

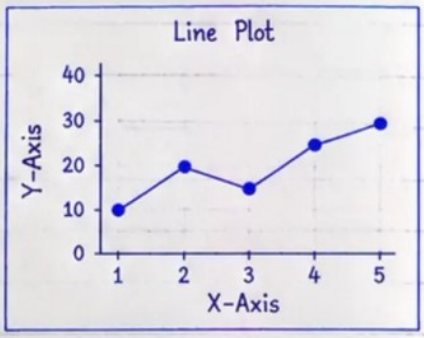
- We can install Matplotlib using pip.
- We also import pyplot for plotting graphs.

```
pip install matplotlib                  # install matplotlib
import matplotlih.pyplot as plt         # import pyplot
```

2. Line Plot

- Used to show trends over time.
- Useful for continuous data.

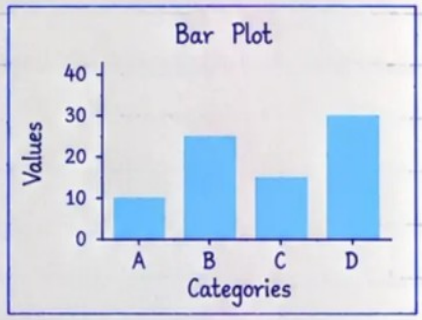
```
x = [1, 2, 3, 4, 5]                     # x values
y = [10, 20, 15, 25, 30]                # y values
plt.plot(x, y, marker="o")              # create line plot
plt.title("Line Plot")                  # set title
plt.xlabel("X-Axis")                    # x-axis label
plt.ylabel("Y-Axis")                    # y-axis label
plt.show()                              # show the plot
```



3. Bar Plot

- Used to compare categorical data.
- Useful for showing counts or frequency.

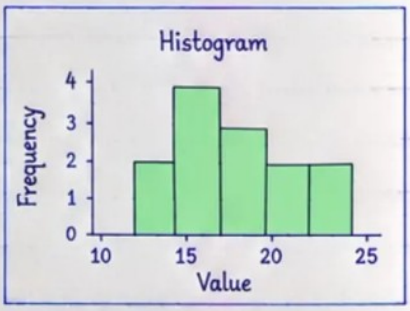
```
categories = ['A', 'B', 'C', 'D']                   # categories
values = [10, 25, 15, 30]                           # values
plt.bar(categories, values, color="skyblue")        # create bar plot
plt.title("Bar Plot")                               # set title
plt.xlabel("Categories")                            # x-axis label
plt.ylabel("Values")                                # y-axis label
plt.show()                                          # show the plot
```



4. Histogram

- Used to show the distribution of numerical data.
- Helps to understand the frequency of values.

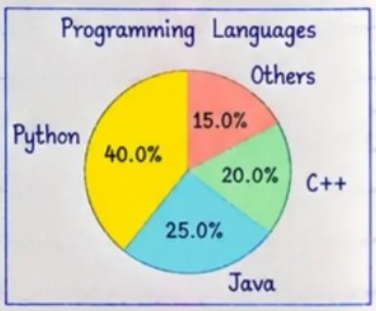
```
data = [12, 15, 14, 10, 18, 20, 15, 16, 22, 24]                 # data values
plt.hist(data, bins=5, color='lightgreen', edgecolor='black')   # create histogram
plt.title("Histogram")                                          # set title
plt.xlabel("Value")                                             # x-axis label
plt.ylabel("Frequency")                                         # y-axis label
plt.show()                                                      # show the plot
```



5. Pie Chart

- Used to show proportion of categories.
- Useful for percentage distribution.

```
labels = ['Python', 'Java', 'C++', 'Others']                    # categories
sizes = [40, 25, 20, 15]                                        # values
plt.pie(sizes, labels=labels, autopct='%1.1f%%',                # create pie chart
    colors=['gold', 'skyblue', 'lightgreen', 'salmon'])
plt.title("Programming Languages")                              # set title
plt.show()                                                      # show the plot
```



6. Key Points

- Matplotlib provides many types of plots (line, bar, histogram, pie, etc.).
- We can customize plots using title, labels, colors and markers.
- Useful for data analysis, reporting and presentations.

DATE:

Real-World Data Analysis Project using Python

In this page, we will work on a complete data analysis project step by step. We will use a real-world dataset and perform data cleaning, exploration, visualization and draw useful insights.

1. Project Overview

- **Dataset** : Students Performance Dataset (marks, study hours, etc.)
- **Goal** : Find how study hours, gender and other factors affect student marks.
- **Steps** : Load data → Clean data → Explore data → Visualize data → Draw insights.

2. Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns                    # for better visualizations
plt.style.use('seaborn-v0_8")            # set plot style
```

3. Load the Dataset

```
df = pd.read_csv("students.csv")         # read CSV file
print(df.head())                         # show first 5 rows
print(df.shape)                          # number of rows and columns
print(df.info())                         # dataset information
```

Sample Data (first 5 rows)

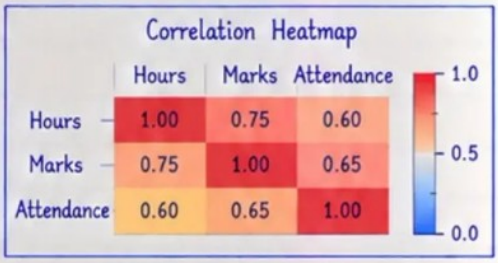
Gender	StudyHours	Marks	Attendance
Male	2.5	65	78
Female	3.0	78	85
Male	4.0	70	80
Female	5.5	90	92
Male	1.0	50	60

4. Data Cleaning

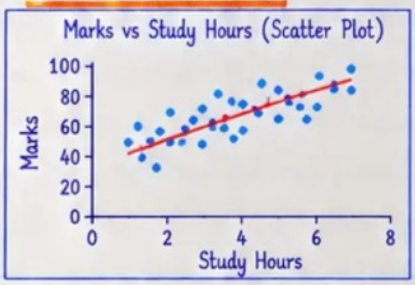
```
print(df.isnull().sum())                 # check missing values
df = df.dropna()                         # remove missing values
df = df.drop_duplicates()                # remove duplicate rows
df['Marks'] = df['Marks'].astype(int)    # change data type
df.rename(columns={'StudyHours' : 'Hours'}, inplace=True)
print(df.head())
```

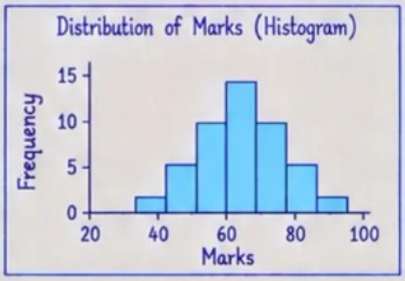
5. Exploratory Data Analysis (EDA)

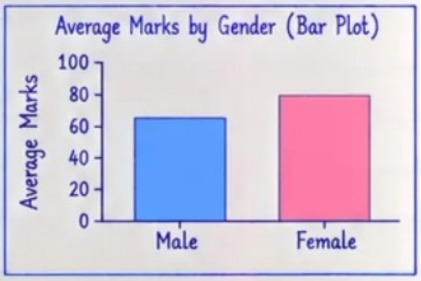
```
print(df.describe())                     # summary statistics
print(df['Marks'].mean())                # average marks
print(df['Marks'].median())              # median marks
print(df['Marks'].std())                 # standard deviation
print(df.corr())                         # correlation between columns
```



6. Data Visualization







7. Insights from the Analysis

- There is a positive correlation between study hours and marks.
- Female students have higher average marks than male students.
- Attendance also has a moderate positive correlation with marks.
- Students who study more than 5 hours generally score above 80.

8. Key Points

- A complete data analysis project includes data cleaning, EDA, visualization and insights.
- Real-world datasets may have missing values and outliers.
- Python libraries like Pandas, Matplotlib and Seaborn make analysis easy and powerful.

DATE:

Machine Learning Basics in Python (Part - 1)

Machine Learning (ML) is a branch of Artificial Intelligence that allows computers to learn from data and make predictions without being explicitly programmed. In this page, we will learn the basics and build our first ML model using Python.

1. What is Machine Learning ?

- ML finds patterns in data and makes predictions.
- It improves performance with more data.
- Common applications : prediction, classification, recommendation, etc.
- Types :
  - **Supervised Learning** (uses labeled data)
  - **Unsupervised Learning** (finds hidden patterns)
  - **Reinforcement Learning** (learns from rewards)

2. Installing and Importing Required Libraries

We will use scikit-learn, a popular Python library for machine learning.

```
# install scikit-learn (run in terminal)
pip install scikit-learn

# import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

Library Purpose :

- NumPy : numerical operations
- Pandas : data handling
- Matplotlib : data visualization
- Scikit-learn : machine learning algorithms

3. Sample Dataset

We will use a simple dataset of Study Hours vs Marks to predict marks using Linear Regression.

```
# create sample dataset
data = {
    'Hours' : [1, 2, 3, 4, 5],
    'Marks' : [20, 40, 55, 70, 85]
}
df = pd.DataFrame(data)
print(df)
```

Output :

	Hours	Marks
0	1	20
1	2	40
2	3	55
3	4	70
4	5	85

Key Points :

- Machine learning helps us predict values from data.
- We need libraries like NumPy, Pandas, Matplotlib and Scikit-learn.
- We start with a simple dataset to understand the process.
- In next page (Part - 2), we will train the model and make predictions.

DATE:

Machine Learning Basics in Python (Part - 2)

1. Preparing Data for Machine Learning

- Before training a model, we need to split the data into training and testing sets.
- Training data is used to learn the pattern.
- Testing data is used to check the performance of the model.

```
# import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split

# create dataset
data = {
    'Hours' : [1, 2, 3, 4, 5],
    'Marks' : [20, 40, 55, 70, 85]
}
df = pd.DataFrame(data)
X = df[['Hours']]   # input feature (2D array)
y = df['Marks']     # target variable

# split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(X_train)
print(X_test)
```

Key Points :

- We split data into training (80%) and testing (20%).
- X contains input features (Hours).
- y contains target values (Marks).
- random_state is used to get same result every time.

Output (Example) :

X_train :

Hours
3
1
5
4

X_test :

Hours
2

2. Training a Linear Regression Model

- Linear Regression is a simple machine learning algorithm used to predict a continuous value.
- It finds the best straight line that fits the data.

```
# import model and evaluation metric
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# create and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# make predictions
y_pred = model.predict(X_test)

# evaluate the model
print("Predicted Marks :", y_pred)
print("Actual Marks :", y_test.values)
print("R2 Score :", r2_score(y_test, y_pred))
print("Mean Squared Error :", mean_squared_error(y_test, y_pred))
```

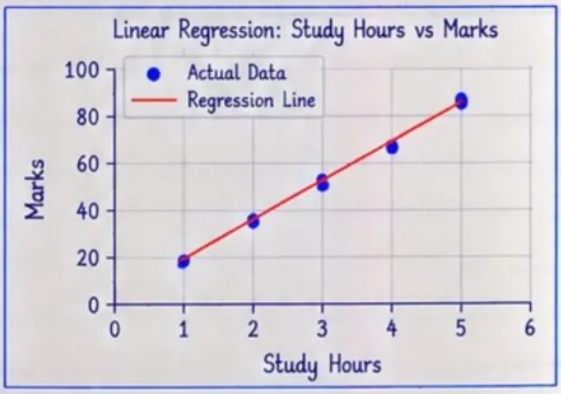
Output (Example) :

```
Predicted Marks : [42.5]
Actual Marks : [40]
R2 Score : 0.98
Mean Squared Error : 6.25
```

3. Visualizing the Regression Line

- We can plot the actual data points and the regression line to see how well the model fits.
- The red line shows the predicted values.

```
# visualize the data and regression line
import matplotlib.pyplot as plt
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Linear Regression: Study Hours vs Marks")
plt.legend()
plt.grid(True)
plt.show()
```



4. Key Takeaways

- We learned how to split data, train a Linear Regression model and make predictions.
- We also visualized the regression line.

Machine Learning Basics in Python (Part - 3)

1. Model Evaluation Metrics

- After training a model, we need to check how well it performs.
- Different metrics are used for different types of problems.
- For Regression, we use metrics like Mean Squared Error (MSE) and R^2 Score.
- For Classification, we use metrics like Accuracy, Precision, Recall and F1-Score.

Types of Metrics :

- Regression Metrics
  - Mean Squared Error (MSE)
  - Mean Absolute Error (MAE)
  - R^2 Score
- Classification Metrics
  - Accuracy
  - Precision
  - Recall
  - F1-Score

2. Regression Metrics (Example)

- We will calculate Mean Squared Error (MSE), Mean Absolute Error (MAE) and R^2 Score for our Linear Regression model.

```
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score
# assume y_test and y_pred are already available
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error :", mse)
print("Mean Absolute Error :", mae)
print("R2 Score :", r2)
```

Output (Example) :

```
Mean Squared Error  : 6.25
Mean Absolute Error : 2.10
R2 Score            : 0.98
```

Interpretation :

- Lower MSE and MAE means better model.
- R^2 Score ranges from 0 to 1.
- R^2 = 1 means perfect prediction.
- R^2 closer to 0 means poor performance.

3. Classification Metrics (Example)

- We will use a classification example to understand Accuracy, Precision, Recall and F1-Score.

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix
# assume y_true and y_pred are available
acc = accuracy_score(y_true, y_pred)
pre = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
cm = confusion_matrix(y_true, y_pred)
print("Accuracy :", acc)
print("Precision :", pre)
print("Recall :", rec)
print("F1-Score :", f1)
print("Confusion Matrix :\n", cm)
```

Output (Example) :

```
Accuracy  : 0.90
Precision : 0.92
Recall    : 0.88
F1-Score  : 0.90
Confusion Matrix :
 [[50  5]
  [ 3 42]]
```

Formula for F1-Score :

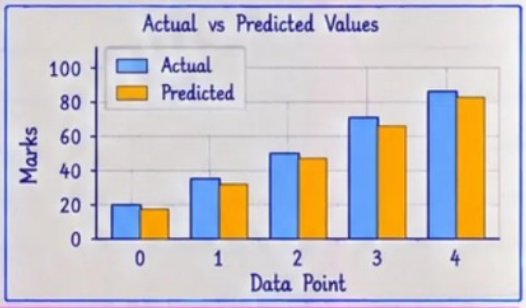
$$F1 = 2 \times \frac{(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

- F1-Score gives a balance between precision and recall.

4. Comparison of Regression Metrics (Visualization)

- We can compare actual vs predicted values using a bar plot.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(len(y_test))
plt.bar(x - 0.2, y_test, width=0.4, label='Actual', color='skyblue')
plt.bar(x + 0.2, y_pred, width=0.4, label='Predicted', color='orange')
plt.xlabel("Data Point")
plt.ylabel("Marks")
plt.title("Actual vs Predicted Values")
plt.legend()
plt.show()
```



Key Takeaways :

- We learned different evaluation metrics for regression and classification.
- Regression → MSE, MAE, R^2 Score.
- Classification → Accuracy, Precision, Recall, F1-Score.
- Metrics help us understand how well our model is performing.

DATE:

Probability in Python for Data Analysis

Probability helps us to understand the likelihood of an event. It is useful in data analysis, decision making and machine learning.

1. Introduction to Probability

- Probability is the measure of how likely an event is to occur.
- It always lies between 0 and 1.
- P(Event) = Number of favorable outcomes / Total number of outcomes

Example :

A dice is rolled. Find the probability of getting a 3.
Total outcomes = 6 (1, 2, 3, 4, 5, 6)
Favorable outcomes for 3 = 1
P(3) = 1/6 = 0.1667

Key Points :

- $0 \leq P(E) \leq 1$
- P(Sure Event) = 1
- P(Impossible Event) = 0
- P(A or B) = P(A) + P(B) - P(A and B)

2. Basic Probability with Python

- We can use simple Python code to calculate probability.
- Useful for simulations and random experiments.

```
import random                          # import random module
roll = random.randint(1, 6)            # simulate dice roll
print("Dice Roll :", roll)
# probability of getting 3
prob = 1/6
print("Probability of 3 :", prob)
```

Sample Output :

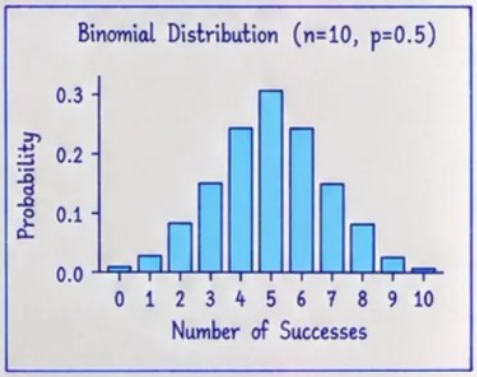
```
Dice Roll : 4
Probability of 3 : 0.1666666667
```

3. Probability Distribution

- A probability distribution shows all possible values and their probabilities.
- Common distributions : Binomial, Normal, Poisson, etc.

Example : Binomial Distribution

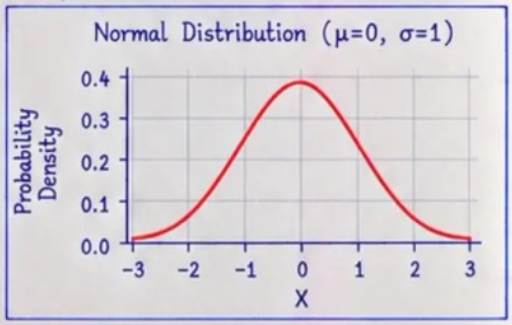
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom
n = 10                                 # number of trials
p = 0.5                                # probability of success
x = np.arange(0, n+1)
y = binom.pmf(x, n, p)
plt.bar(x, y, color="skyblue", edgecolor="black")
plt.title("Binomial Distribution (n=10, p=0.5)")
plt.xlabel("Number of Successes")
plt.ylabel("Probability")
plt.show()
```



4. Normal Distribution

- A normal distribution is a continuous probability distribution.
- It is widely used in real-world data (like height, marks, etc.).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
x = np.linspace(-3, 3, 100)
y = norm.pdf(x, 0, 1)
plt.plot(x, y, color="red")
plt.title("Normal Distribution (μ=0, σ=1)")
plt.xlabel("X")
plt.ylabel("Probability Density")
plt.grid(True)
plt.show()
```



5. Key Points

- Probability helps us to quantify uncertainty.
- Distributions help us to model real-world data.

DATE:

Introduction to Python for Data Analysis

Python is a powerful and easy to learn programming language. It is widely used in data analysis because of its simple syntax and useful libraries like Pandas, NumPy, Matplotlib and Seaborn.

1. Why Use Python for Data Analysis ?

- Simple and easy to learn.
- Large number of libraries for data analysis.
- Can handle small and large datasets.
- Useful for data cleaning, analysis and visualization.
- Widely used in industry and research.



2. Installing Python

- Download Python from official website : https://www.python.org
- During installation, check the option "Add Python to PATH".
- We can use Jupyter Notebook for writing and running code.

3. Checking Python Installation

```
python --version
# Output : Python 3.12.0 (example)
```

```
print("Hello, World!")
# Output : Hello, World!
```

4. Important Libraries for Data Analysis

- NumPy - For numerical calculations and arrays.
- Pandas - For data cleaning and analysis.
- Matplotlib - For data visualization (basic plots).
- Seaborn - For advanced and attractive visualizations.

5. Installing Libraries using pip

```
pip install numpy pandas matplotlib seaborn
```

Install multiple libraries at once

6. Importing Libraries in Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# NumPy library
# Pandas library
# Matplotlib for plotting
# Seaborn for advanced plots
```

DATE:

Logistic Regression in Python (Part - 4)

Logistic Regression is a popular classification algorithm used to predict categorical outcomes (Yes/No, 0/1, True/False) based on input features.

1. Working of Logistic Regression

- It uses a sigmoid function to convert the output into a probability between 0 and 1.
- If probability ≥ 0.5 → class 1 (Positive)
- If probability < 0.5 → class 0 (Negative)
- Common applications : spam detection, disease prediction, pass/fail, etc.

Sigmoid Function :

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Output range : 0 to 1

2. Example Dataset (Student Pass/Fail)

- We will use a simple dataset to predict whether a student will pass (1) or fail (0) based on the number of study hours.

```
# create sample dataset
import pandas as pd
data = {
    'Hours' : [1, 2, 3, 4, 5, 6, 7, 8],
    'Pass'  : [0, 0, 0, 0, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
print(df)
```

Sample Data :

Hours	Pass (0 = Fail, 1 = Pass)
1	0
2	0
3	0
4	0
5	1
6	1
7	1
8	1

3. Training the Logistic Regression Model

- We will split the data, train the model and then make predictions.
- We use LogisticRegression from sklearn.

```
# import required libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X = df[['Hours']]   # input feature
y = df['Pass']      # target variable

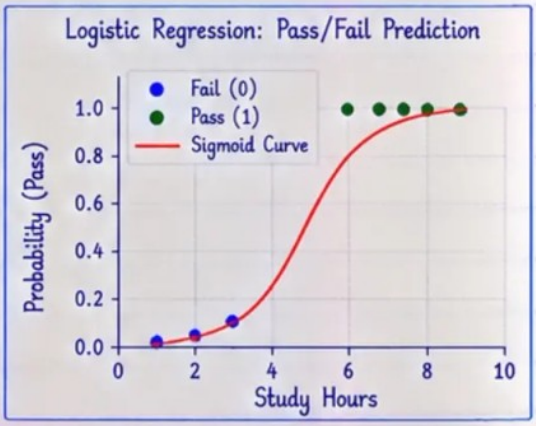
# split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# create and train model
model = LogisticRegression()
model.fit(X_train, y_train)

# make predictions
y_pred = model.predict(X_test)
print("Predicted values :", y_pred)
print("Actual values    :", y_test.values)
print("Accuracy :", accuracy_score(y_test, y_pred))
```

Output (Example) :

```
Predicted values : [1 0]
Actual values    : [1 0]
Accuracy : 1.0
```



4. Key Points

- Logistic Regression is used for classification problems.
- It predicts probability using sigmoid function.
- If probability ≥ 0.5 → class 1, else class 0.
- We use LogisticRegression from sklearn.
- Common metrics : Accuracy, Precision, Recall, F1-Score